

CodeForge

Co-team, не co-pilot

Команда AI-агентов, которая берёт спецификацию на естественном языке и производит работающий код — с встроенным аудитом, конкуренцией моделей и автоматической сходимостью.

Проблема

Современные AI-инструменты для разработки = **AI-помощник одному программисту**.

- Одна модель. Один проход. Ручной code review.
- Скрытое смещение: ты заперт в стиле одного LLM (Claude **или** GPT **или** Gemini).
- Нет встроенного контроля качества — нужно вручную просить «теперь поищи баги».
- Каждая попытка — **чёрный ящик**. Что попробовал? Где провалился? Почему именно так?

Чем сложнее задача, тем хуже работает «один умный помощник».

Инсайт

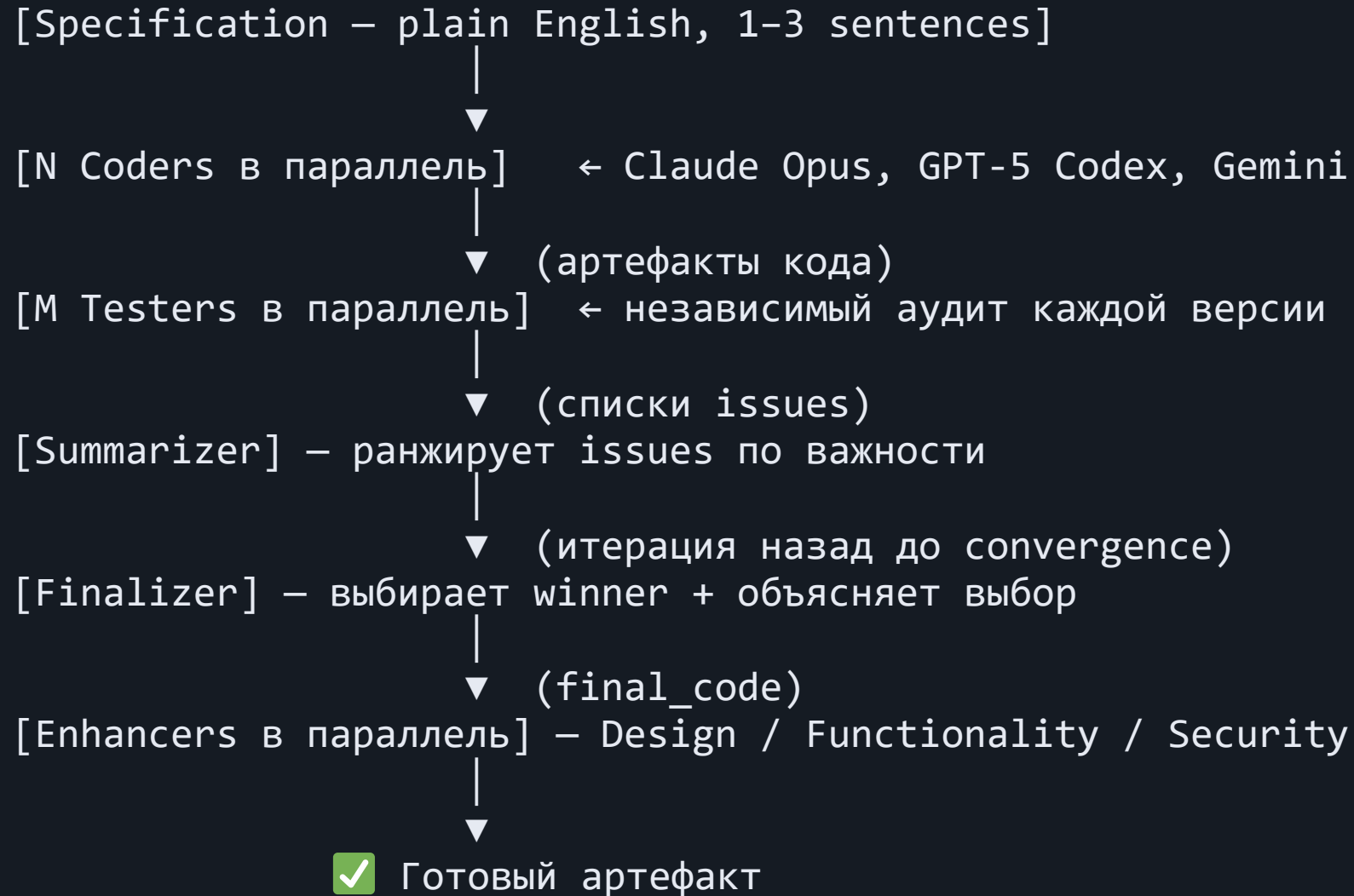
В человеческой разработке мы знаем что:

- **Два инженера думают по-разному** → парное программирование, peer review.
- **Тестировщик ловит то, что не видит автор** → роль QA.
- **Архитектор оценивает иначе, чем кодер** → review на разных уровнях.

| Почему AI-кодинг должен быть исключением?

CodeForge приносит **командную динамику** в AI-генерацию кода.

Решение: Multi-Agent Orchestration



Как это выглядит вживую

- **Real-time граф агентов** — видишь как каждый агент стримит код
- **Метрики:** токены, стоимость, итерации, время
- **WebSocket-события** — каждый шаг записан и replay'ится
- **Side-panels** — открой любого агента, посмотри что он написал

Не «сгенерируй код за 30 секунд». **Покажи как.**

Wisdom of crowds

Модель	Что приносит
Claude Opus 4.5	Тщательность, длинный контекст, объяснения
GPT-5 Codex	Скорость, краевые случаи, оптимизации
Gemini 2.5 Pro	Альтернативный архитектурный взгляд

В одной сессии — все три модели независимо решают задачу.

Finalizer сравнивает результаты и выбирает лучший (с объяснением).

Это не «*mixture of experts*» на уровне токенов. **Это competition на уровне решений.**

Built-in Quality Assurance

В большинстве AI-инструментов QA — это **ТЫ**.

В CodeForge:

Этап	Кто	Что делает
Generation	Coders	Параллельно пишут код от спецификации
Audit	Testers	Независимо ищут баги в каждой версии
Prioritization	Summarizer	Ранжирует issues: critical → low
Iteration	Coders v2	Учитывают summary, переписывают
Selection	Finalizer	Выбирает финальную версию + README
Polish	Enhancers	Design / Functionality / Security passes

До сходимости. Без супервизии.

Replay & Share

Каждая сессия — **first-class artifact**:

- 🎬 Pre-recorded demos с покадровой нарративной плашкой
- 🔗 Deep-links в конкретные главы (`?startAtChapter=N`)
- ⏮ Speed control (0.5× — 60×), keyboard shortcuts, mobile-ready
- 📤 Share link — любая сессия публикуется как marketing demo

Use case: показал заказчику demo замысла → подписал contract → запустил session с реальной спекой.

Один путь от идеи через показ до рабочего кода.

Реальные примеры из коробки

Demo	Что внутри	Время
 Mandelbulb 3D Attractor	WebGL2 ray-marching, 8 палитр, морфинг $n=4 \rightarrow 20$	~3 мин
 Galaxy Collision (Barnes-Hut)	5000 particles, gravitational N-body	~3 мин
 Game of Life	Pattern gallery (Glider, Gosper), play controls	~2 мин
 Falling Sand Sandbox	8 elements (sand, water, fire), 60fps	~3 мин
 Gray-Scott Reaction-Diffusion	WebGL2 fragment shader, parameter sliders	~3 мин

Каждое — single-page HTML. Открывается прямо из браузера. Никаких зависимостей.

Когда CodeForge — **правильный выбор**

- ✓ Спецификация формулируется в 1–3 предложениях
- ✓ Результат — 1–2 файла, single page
- ✓ Алгоритмика / визуализация / симуляция
- ✓ Educational / dem'нстрационный материал
- ✓ A/B-сравнение моделей на одной задаче
- ✓ Нужен **audit trail** «кто что сделал»
- ✓ Качество > скорость (готовы тратить 3–10 мин)

Sweet spot: прототип за вечер с гарантией что прошёл tester-аудит.

Когда НЕ CodeForge

- ✗ Edit'ы в большой существующей кодовой базе → **Claude Code / Cursor**
- ✗ Multi-file рефакторы → **Cursor Composer**
- ✗ Real-time pair-programming → **Cursor / Windsurf**
- ✗ Inline tab-completion → **Copilot**
- ✗ Bug fix в конкретной строке файла → **Claude Code** (overhead неоправдан)
- ✗ Architecture discussion → **ChatGPT / Claude (chat)**

CodeForge — не замена этим инструментам. **Это другой жанр.**

CodeForge vs. остальные

	CodeForge	Claude Code	Cursor	Codex	v0 / Bolt	Copilot
Multi-agent	✓	✗	✗	✗	✗	✗
Multi-model	✓	✗	⚠	✗	✗	✗
Built-in QA	✓	⚠	⚠	⚠	✗	✗
Iteration loop	✓	manual	manual	manual	manual	—
Audit trail	✓	⚠	⚠	✓	✗	✗
Live viz	✓	✗	inline	✗	preview	ghost
Replay/Share	✓	✗	✗	PR	URL	✗
Greenfield	✓	⚠	⚠	⚠	✓	✗
Existing codebase	✗	✓	✓	✓	✗	✓

Hybrid workflows

CodeForge **дополняет**, а не заменяет.

Шаг 1	Шаг 2
CodeForge генерирует standalone particle-эффект	Cursor интегрирует в React-компонент landing-page
CodeForge пишет force-directed graph algorithm с тестами	Claude Code встраивает в <code><GraphView /></code> дашборда
CodeForge делает sandbox HTML с iframe-securing	Claude Code оборачивает как route в production app
CodeForge строит SVG-лого с morphing-анимацией	Claude Code упаковывает как <code><Logo /></code> компонент

Принцип: CodeForge для greenfield-куска, IDE-tool для интеграции.

Стоимость единицы качества

Tool	\$ за задачу	Что получаешь
CodeForge	\$0.30 – \$2.00	Артефакт + audit trail + Tester-аудит
Claude Code	\$0.05 – \$0.50	Diff, требует ревью человеком
OpenAI Codex	\$0.10 – \$1.00	PR, требует ревью человеком
Cursor	\$0.20 – \$1.50	Inline-помощь, требует постоянного supervision
v0 / Bolt	\$0.05 – \$0.30	UI-page, не для production
Copilot	flat subscription	Tab-completion

CodeForge дороже **на задачу** — но почти **не требует супервизии**.

Удельная стоимость качества часто **ниже** в sweet-spot задачах.

Технологии

Frontend: React 19 + Vite + TypeScript + Tailwind + React Flow (live graph)

Backend: FastAPI + SQLAlchemy 2 + Alembic + Postgres + Celery + Redis

Streaming: WebSocket с reconnect/backoff

Sandbox: Headless Chromium (Playwright) для JS, Docker для Python

LLM-providers: Anthropic, OpenAI, Google, OpenRouter

Что внутри audit trail

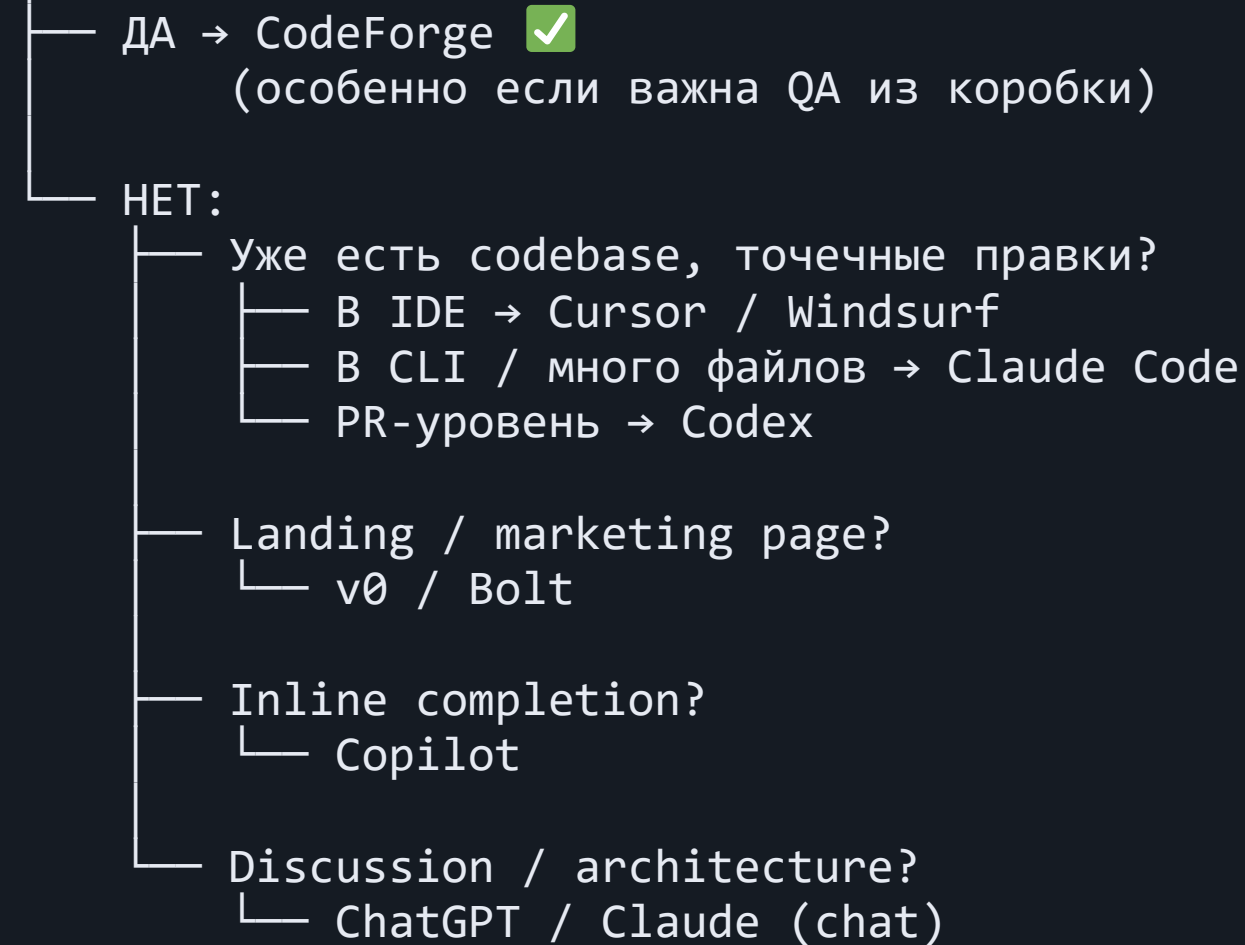
Полная запись WebSocket-событий каждой сессии:

```
t=0.0    workflow_started
t=14.0    iteration 1 → phase: coding
t=14.2    agent_started: coder_0 (claude-opus-4.5)
t=15.0    agent_streaming: coder_0 → <!DOCTYPE html>...
...
t=68.4    agent_completed: coder_0 — 1850 tokens, $0.12, 0 issues
t=68.9    agent_started: tester_0 (claude-sonnet-4.5)
...
t=158.0   finalizer_completed → chose coder_1 (reasoning: "Cleaner sphere-tracing loop, ...")
t=160.0   workflow_completed
```

Идеально для **post-mortem**, **обучения**, **billing-аудита**, **демо**.

Decision-tree за 10 секунд

Задача greenfield, single-page, спека в 1-3 предложениях?



Дорожная карта

Q3 2025

-  Multi-coder ensemble, Testers, Summarizer, Finalizer
-  Enhancement phase (Design / Functionality / Security)
-  Replay & deep-link demos

Q4 2025

-  WS reconnect UI, retry-agent affordance
-  Multi-file output поддержка
-  Sessions API + CLI client

Q1 2026

-  Git integration: PR-based output, repo-aware tester
-  Domain-specific agent personalities (security-first, perf-first)
-  Custom agent roles via config

H2 2026

-  On-prem edition
-  Fine-tunable role-specific models

Попробуй сейчас

 stage.gotcode.ai/demos — 5 готовых демо, играй прямо в браузере

 gotcode.ai — создай свою первую сессию

 gotcode.ai/docs — спецификации, API, примеры

Best way to predict the future is to invent it.

— *Alan Kay*

Спасибо

CodeForge — co-team, not co-pilot.

Вопросы, демо-сессия 1-on-1, partnership inquiries: gotcode.ai/contact